UNIVERSIDAD PRIVADA FRANZ TAMAYO

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA EN SISTEMAS



EXAMEN PRÁCTICO HITO 3

PRUEBAS DE CAJA NEGRA, CAJA BLANCA, GHERKIN Y REGISTRO DE DEFECTOS

CASO: "SAUCEDEMO (SWAG LABS)"

ESTUDIANTE:

- Jhorddy Danner Condori Mamani

MATERIA:

- Testing, Quality Assurance

DOCENTE:

- Ing. Reynel Sanchez Chuquimia

REPOSITORIO: https://github.com/DannerCM/Examen-Hito-3-TQA

LA PAZ - BOLIVIA

GESTIÓN II - 2026

Índice

1. Pruebas de caja negra 1
1.1. Funcionalidades y alcance 1
1.2. Criterios de aceptación 1
1.3. Técnicas de diseño aplicadas 2
2. Casos de prueba 3
3. Ejecución 8
4. Defectos y resultados 17
4.1. Resumen de ejecución 17
4.2. Defectos encontrados 17
4.3. Métricas 17
4.4. Matriz de trazabilidad 18
4.5. Análisis de los resultados 18
4.6. Reporte del defecto BUG-01-F 19
5. Análisis de caja blanca 20
5.1. Decisiones y condiciones simples 20
5.2. Criterio de cobertura 20
5.3. Mínimo de casos 20
5.4. Casos diseñados 21
5.5. Ajuste necesario 21
5.6. Valores límite 22
6. Escenarios Gherkin 23
6.1. Inicio de sesión 23
6.2. Checkout 23
6.3. Relación con los casos de prueba 24
7. Organización en Git y GitHub 25
7.1. Estructura del repositorio 25
7.2. Comandos utilizados 25
7.3. Issues y tablero 25
8. Conclusiones 27
9. Recomendaciones 27
10. Referencias 28

1. Pruebas de caja negra

El presente documento registra el examen práctico del Hito 3 de la materia. Las pruebas se aplicaron sobre SauceDemo (Swag Labs), una tienda en línea que Sauce Labs publica para practicar pruebas funcionales. Se diseñaron cinco casos de caja negra sobre tres funcionalidades, se ejecutaron sobre el sitio en funcionamiento y se analizaron sus resultados. De esa ejecución surgió un defecto de severidad alta.

El documento incluye además el análisis de caja blanca de un fragmento de código Python, los escenarios en Gherkin y la organización del trabajo en GitHub. La ejecución se realizó el 20 de octubre de 2026 sobre la versión pública del sitio.

1.1. Funcionalidades y alcance

La selección siguió tres criterios: impacto económico, riesgo de la operación y frecuencia de uso. Se priorizó el checkout, que es el paso donde se concreta la venta, y a continuación el carrito y el acceso a la tienda.

ID	Funcionalidad	Módulo	Casos
F-01	Inicio de sesión	Login	TC-01, TC-02
F-02	Gestión del carrito de compras	Carrito	TC-03
F-03	Datos del comprador y confirmación de compra	Checkout	TC-04, TC-05

Tabla 1. Funcionalidades seleccionadas para las pruebas de caja negra
Fuente: Elaboración propia

Quedaron fuera del alcance los usuarios de demostración que simulan fallas a propósito (problem_user, error_user, performance_glitch_user y visual_user), el ordenamiento del catálogo y el menú lateral. Tampoco se incluyen pruebas no funcionales, conforme a la consigna del examen.

1.2. Criterios de aceptación

F-01. Inicio de sesión:

- Un usuario habilitado accede al catálogo con sus credenciales.
- Un usuario bloqueado no accede y recibe un mensaje que indica el bloqueo.

F-02. Carrito:

- Agregar un producto incrementa el contador del carrito.
- Retirar un producto lo elimina de la lista y reduce el contador.

F-03. Checkout:

- El formulario no avanza mientras falte un dato obligatorio e indica cuál falta.
- Una compra sin productos no puede confirmarse.

1.3. Técnicas de diseño aplicadas

La técnica se eligió según la forma de la regla que debía verificarse. Donde las entradas se agrupan en clases con el mismo comportamiento se aplicó partición de equivalencia. Donde el comportamiento cambia en un umbral se aplicaron valores límite.

Técnica	Casos	Justificación
Partición de equivalencia	TC-01, TC-02, TC-03, TC-04	Las entradas forman clases de comportamiento idéntico: usuario habilitado o bloqueado, campo completo o vacío, producto agregado o retirado
Valores límite	TC-05	La cantidad de productos del carrito tiene su límite inferior en cero, el punto donde la compra debería dejar de ser posible

Tabla 2. Técnicas de diseño aplicadas por caso
Fuente: Elaboración propia

2. Casos de prueba

Se diseñaron cinco casos, dos positivos y tres negativos. El resultado esperado de cada caso quedó definido antes de su ejecución. Los negativos tienen más peso en el conjunto porque los errores suelen aparecer cuando el usuario hace algo que el sistema no espera, y el defecto encontrado lo confirma.

Caso TC-01

ID DEL TC	TC-01	**PRIORIDAD**	Alta
TÍTULO	Inicio de sesión con credenciales válidas	**TIPO**	Funcional positiva
MÓDULO	Login	**AMBIENTE**	Producción pública (demo)
¿QUÉ SE QUIERE VALIDAR?	Verificar que un usuario válido accede al catálogo de productos.		
Estado inicial	Página de login abierta, sin sesión iniciada.		
Usuario / permisos	standard_user (usuario de demostración habilitado).		
Configuración	Chromium 156, resolución 1280x800, sin extensiones.		

Dato	**Valor**	**Origen**	**Observación**
Usuario	standard_user	Ambiente	Publicado en la página de login
Contraseña	secret_sauce	Ambiente	Común a todos los usuarios

#	**Paso / acción**	**Dato utilizado**	**Resultado esperado**
1	Escribir el usuario en Username	standard_user	El campo muestra el usuario
2	Escribir la contraseña en Password	secret_sauce	El campo oculta los caracteres
3	Pulsar Login	-	Se carga /inventory.html con el título Products

CRITERIO DE ÉXITO	El usuario entra al catálogo de productos.

Tabla 3. Caso de prueba TC-01
Fuente: Elaboración propia

Caso TC-02

ID DEL TC	TC-02	**PRIORIDAD**	Alta
TÍTULO	Inicio de sesión con usuario bloqueado	**TIPO**	Funcional negativa
MÓDULO	Login	**AMBIENTE**	Producción pública (demo)
¿QUÉ SE QUIERE VALIDAR?	Verificar que un usuario bloqueado no ingresa y recibe un mensaje claro.		
Estado inicial	Página de login abierta, sin sesión iniciada.		
Usuario / permisos	locked_out_user (usuario de demostración bloqueado).		
Configuración	Chromium 156, resolución 1280x800, sin extensiones.		
Dato	**Valor**	**Origen**	**Observación**
Usuario	locked_out_user	Ambiente	Publicado en la página de login
Contraseña	secret_sauce	Ambiente	Común a todos los usuarios

#	Paso / acción	Dato utilizado	Resultado esperado
1	Escribir el usuario en Username	locked_out_user	El campo muestra el usuario
2	Escribir la contraseña en Password	secret_sauce	El campo oculta los caracteres
3	Pulsar Login	-	Se permanece en el login con un mensaje de usuario bloqueado
CRITERIO DE ÉXITO	El acceso se rechaza y se informa que el usuario está bloqueado.		

Tabla 4. Caso de prueba TC-02
Fuente: Elaboración propia

Caso TC-03

ID DEL TC	TC-03	**PRIORIDAD**	Alta
TÍTULO	Agregar dos productos y eliminar uno desde el carrito	**TIPO**	Funcional positiva
MÓDULO	Carrito	**AMBIENTE**	Producción pública (demo)
¿QUÉ SE QUIERE VALIDAR?	Verificar que el carrito y su contador se actualizan al agregar y eliminar productos.		
Estado inicial	Sesión iniciada y carrito vacío.		
Usuario / permisos	standard_user.		
Configuración	Chromium 156, resolución 1280x800, sin extensiones.		

Dato	**Valor**	**Origen**	**Observación**
Producto 1	Sauce Labs Backpack	Ambiente	USD 29,99
Producto 2	Sauce Labs Bike Light	Ambiente	USD 9,99

#	**Paso / acción**	**Dato utilizado**	**Resultado esperado**
1	Pulsar Add to cart en el primer producto	Sauce Labs Backpack	El contador del carrito muestra 1
2	Pulsar Add to cart en el segundo producto	Sauce Labs Bike Light	El contador muestra 2
3	Abrir el carrito	-	Se listan los dos productos
4	Pulsar Remove en Sauce Labs Bike Light	-	El producto sale de la lista y el contador muestra 1

CRITERIO DE ÉXITO	El carrito contiene solo Sauce Labs Backpack y el contador indica 1.

Tabla 5. Caso de prueba TC-03
Fuente: Elaboración propia

Caso TC-04

ID DEL TC	TC-04	**PRIORIDAD**	Media
TÍTULO	Checkout con campos obligatorios vacíos	**TIPO**	Funcional negativa
MÓDULO	Checkout	**AMBIENTE**	Producción pública (demo)
¿QUÉ SE QUIERE VALIDAR?	Verificar que el formulario de checkout exige First Name, Last Name y Postal Code.		
Estado inicial	Sauce Labs Backpack en el carrito.		
Usuario / permisos	standard_user.		
Configuración	Chromium 156, resolución 1280x800, sin extensiones.		
Dato	**Valor**	**Origen**	**Observación**
First Name	Ana	Generado	Se completa en el paso 3
Last Name	Quispe	Generado	Se completa en el paso 4
Postal Code	(vacío)	Generado	Se deja vacío a propósito

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir el carrito y pulsar Checkout	-	Se carga Checkout: Your Information
2	Pulsar Continue con todo vacío	-	Mensaje Error: First Name is required
3	Completar First Name y pulsar Continue	Ana	Mensaje Error: Last Name is required
4	Completar Last Name y pulsar Continue	Quispe	Mensaje Error: Postal Code is required; no se avanza
CRITERIO DE ÉXITO	El sistema no avanza mientras falte un campo obligatorio e indica cuál falta.		

Tabla 6. Caso de prueba TC-04
Fuente: Elaboración propia

Caso TC-05

ID DEL TC	TC-05	PRIORIDAD	Alta
TÍTULO	Checkout con el carrito vacío	TIPO	Funcional negativa
MÓDULO	Checkout	AMBIENTE	Producción pública (demo)
¿QUÉ SE QUIERE VALIDAR?	Verificar que no se puede completar una compra sin productos en el carrito.		
Estado inicial	Carrito vacío.		
Usuario / permisos	standard_user.		
Configuración	Chromium 156, resolución 1280x800, sin extensiones.		

Dato	Valor	Origen	Observación
First Name	Ana	Generado	-
Last Name	Quispe	Generado	-
Postal Code	0000	Generado	-

#	Paso / acción	Dato utilizado	Resultado esperado
1	Abrir el carrito sin agregar productos	-	El carrito aparece vacío
2	Pulsar Checkout	-	El botón está deshabilitado o se avisa que el carrito está vacío
3	Si se permite avanzar, completar el formulario y pulsar Continue	Ana / Quispe / 0000	No se muestra un resumen sin productos
4	Pulsar Finish	-	No se registra ninguna orden

CRITERIO DE ÉXITO	La compra se bloquea porque no hay productos.

Tabla 7. Caso de prueba TC-05
Fuente: Elaboración propia

3. Ejecución

Los cinco casos se ejecutaron sobre el sitio en funcionamiento mediante un script de Playwright que reproduce las acciones del usuario y guarda una captura en cada paso relevante. El script está en el repositorio, de modo que la ejecución puede repetirse y comprobarse. El conjunto se corrió tres veces y el resultado fue el mismo en las tres.

Fecha de ejecución	20 de octubre de 2026
Aplicación	SauceDemo (Swag Labs), https://www.saucedemo.com/
Versión	Versión pública disponible a la fecha de ejecución
Sistema operativo	Windows 10 Pro
Navegador	Chromium 156.0.8078.4, resolución 1280x800
Herramientas	Playwright 1.64, pytest 9.1, coverage.py, Git y GitHub
Usuarios	standard_user y locked_out_user, contraseña secret_sauce

Tabla 8. Ambiente de ejecución de las pruebas
Fuente: Elaboración propia

Funcionalidad	Casos ejecutados	PASS	FAIL
F-01 Inicio de sesión	2	2	0
F-02 Carrito	1	1	0
F-03 Checkout	2	1	1
Total	5	4	1

Tabla 9. Resultado de la ejecución por funcionalidad
Fuente: Elaboración propia

A continuación se presenta el registro de cada caso con su resultado observado. La evidencia de cada registro aparece después de su tabla, desde la Figura 1 hasta la Figura 9.

TC-01. Inicio de sesión con credenciales válidas

ID DEL TC	TC-01	**EJECUTADO POR**	Jhorddy Danner Condori Mamani
FECHA	20/10/2026	**HORA**	21:15
AMBIENTE	Producción pública (demo)	**VERSIÓN**	Publicada al 20/10/2026
☒ PASS	**☐ FAIL**	**☐ BLOCKED**	**☐ NOT RUN**
¿QUÉ OCURRIÓ REALMENTE?	El sistema abrió https://www.saucedemo.com/inventory.html con el título Products y el ícono del carrito.		
COMPARACIÓN	Coincide con lo esperado.		

#	Paso ejecutado	Resultado observado	Estado	Observación
1	Escribir el usuario en Username	El campo mostró standard_user	PASS	-
2	Escribir la contraseña en Password	El campo ocultó la contraseña	PASS	-
3	Pulsar Login	Se cargó /inventory.html con el título Products	PASS	Seis productos visibles

Tipo	Captura de pantalla
Nombre o referencia	TC-01_login_exitoso
Descripción	Catálogo cargado después del inicio de sesión
DESVIACIÓN ENCONTRADA	No aplica
REFERENCIA DEL DEFECTO	N/A
OBSERVACIONES	Ejecución automatizada con Playwright; resultado repetido en tres corridas.

Tabla 10. Registro de ejecución del caso TC-01
Fuente: Elaboración propia

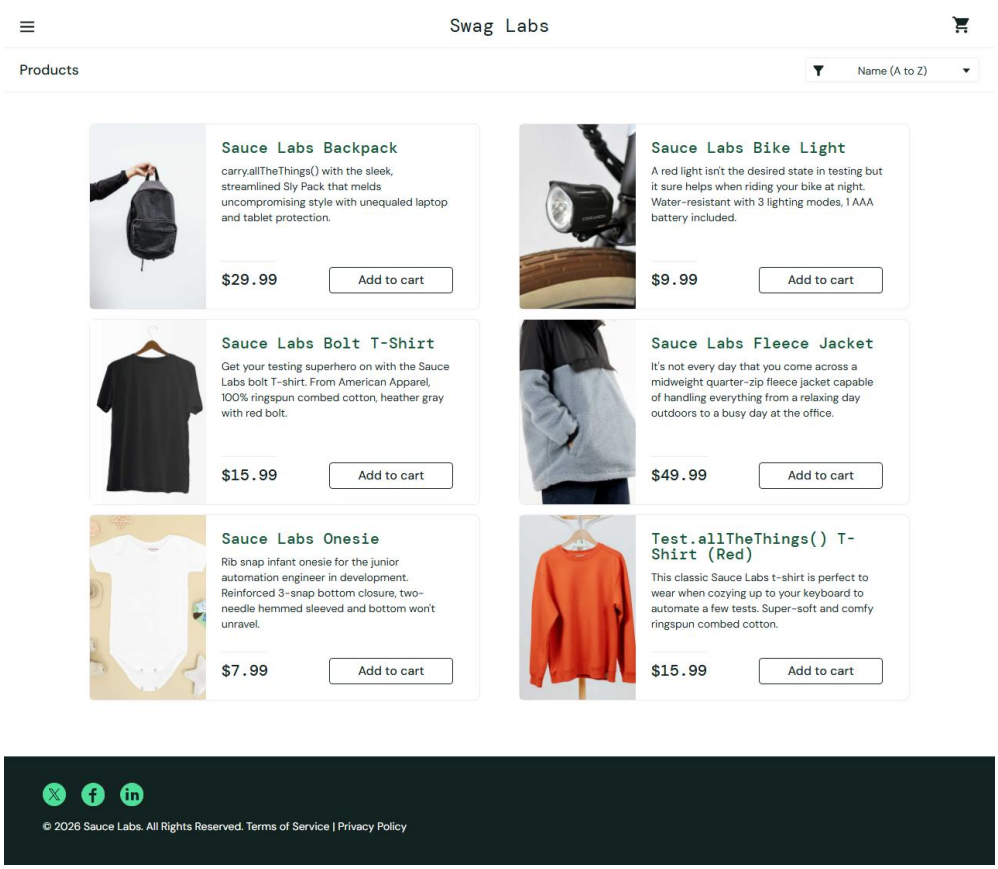


Figura 1. Catálogo cargado después del inicio de sesión (TC-01)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

TC-02. Inicio de sesión con usuario bloqueado

ID DEL TC	TC-02	**EJECUTADO POR**	Jhorddy Danner Condori Mamani
FECHA	20/10/2026	**HORA**	21:15
AMBIENTE	Producción pública (demo)	**VERSIÓN**	Publicada al 20/10/2026
☒ PASS	**☐ FAIL**	**☐ BLOCKED**	**☐ NOT RUN**
¿QUÉ OCURRIÓ REALMENTE?	El sistema permaneció en el login y mostró «Epic sadface: Sorry, this user has been locked out.».		
COMPARACIÓN	Coincide con lo esperado.		

#	Paso ejecutado	Resultado observado	Estado	Observación
1	Escribir el usuario en Username	El campo mostró locked_out_user	PASS	-
2	Escribir la contraseña en Password	El campo ocultó la contraseña	PASS	-
3	Pulsar Login	Mensaje de bloqueo; la URL no cambió	PASS	Campos marcados en rojo

Tipo	Captura de pantalla
Nombre o referencia	TC-02_usuario_bloqueado
Descripción	Mensaje de usuario bloqueado
DESVIACIÓN ENCONTRADA	No aplica
REFERENCIA DEL DEFECTO	N/A
OBSERVACIONES	Ejecución automatizada con Playwright; resultado repetido en tres corridas.

Tabla 11. Registro de ejecución del caso TC-02
Fuente: Elaboración propia

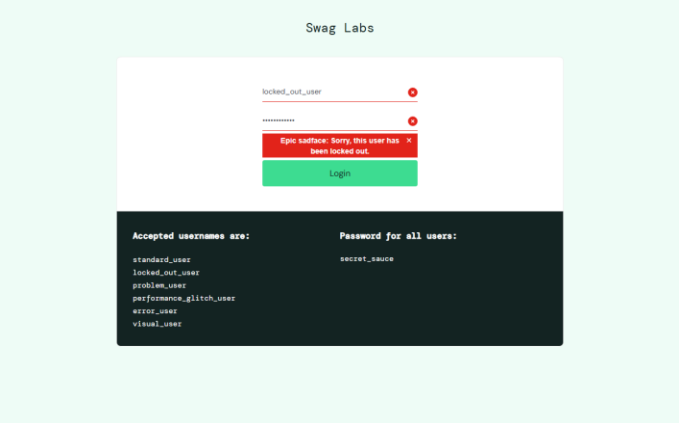


Figura 2. Mensaje de usuario bloqueado (TC-02)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

TC-03. Agregar dos productos y eliminar uno desde el carrito

ID DEL TC	TC-03		**EJECUTADO POR**	Jhorddy Danner Condori Mamani
FECHA	20/10/2026		**HORA**	21:15
AMBIENTE	Producción pública (demo)		**VERSIÓN**	Publicada al 20/10/2026
☒ PASS	**☐ FAIL**		**☐ BLOCKED**	**☐ NOT RUN**
¿QUÉ OCURRIÓ REALMENTE?	El contador pasó de 2 a 1 y el carrito quedó solo con Sauce Labs Backpack.			
COMPARACIÓN	Coincide con lo esperado.			

#	Paso ejecutado	Resultado observado	Estado	Observación
1	Pulsar Add to cart en el primer producto	El contador mostró 1	PASS	-
2	Pulsar Add to cart en el segundo producto	El contador mostró 2	PASS	-
3	Abrir el carrito	Se listaron los dos productos	PASS	-
4	Pulsar Remove en Sauce Labs Bike Light	Quedó un producto y el contador mostró 1	PASS	-

Tipo	2 capturas de pantalla
Nombre o referencia	TC-03a_dos_productos_agregados, TC-03b_producto_eliminado
Descripción	Contador del carrito con dos productos; Carrito con un producto después de eliminar el segundo
DESVIACIÓN ENCONTRADA	No aplica
REFERENCIA DEL DEFECTO	N/A
OBSERVACIONES	Ejecución automatizada con Playwright; resultado repetido en tres corridas.

Tabla 12. Registro de ejecución del caso TC-03
Fuente: Elaboración propia

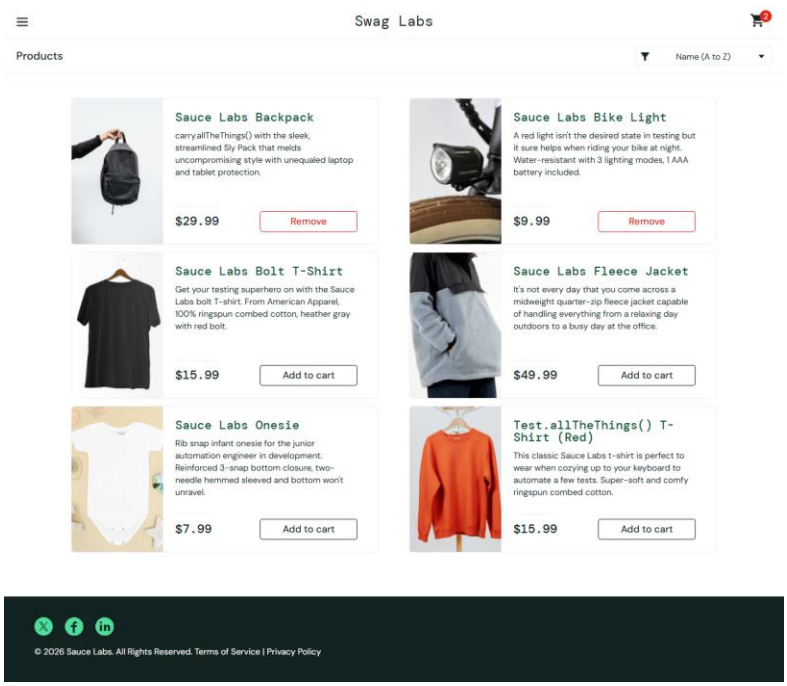


Figura 3. Contador del carrito con dos productos (TC-03)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

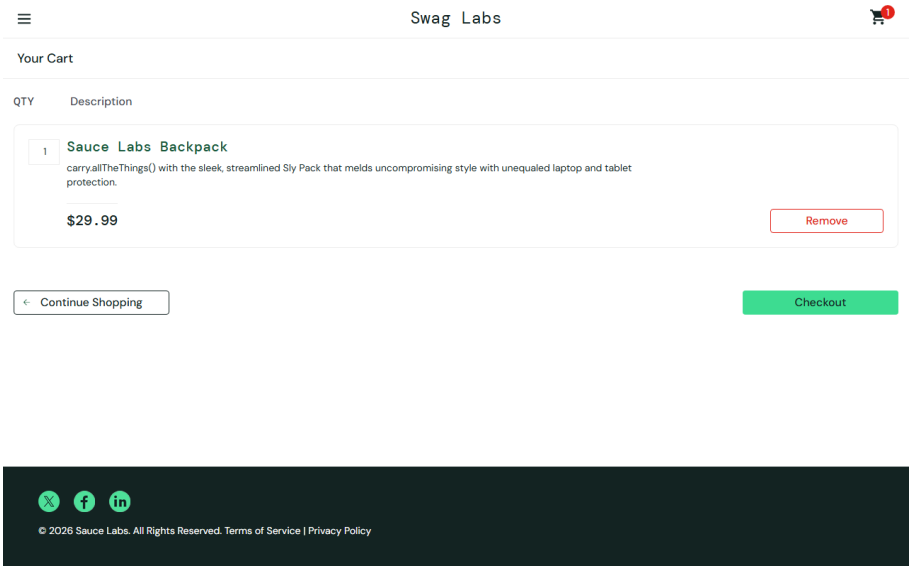


Figura 4. Carrito con un producto después de eliminar el segundo (TC-03)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

TC-04. Checkout con campos obligatorios vacíos

ID DEL TC	TC-04	**EJECUTADO POR**	Jhorddy Danner Condori Mamani
FECHA	20/10/2026	**HORA**	21:15
AMBIENTE	Producción pública (demo)	**VERSIÓN**	Publicada al 20/10/2026
☒ PASS	**☐ FAIL**	**☐ BLOCKED**	**☐ NOT RUN**
¿QUÉ OCURRIÓ REALMENTE?	Se mostraron, en orden, los tres mensajes de error y la página no avanzó.		
COMPARACIÓN	Coincide con lo esperado.		

#	Paso ejecutado	Resultado observado	Estado	Observación
1	Abrir el carrito y pulsar Checkout	Se cargó Checkout: Your Information	PASS	-
2	Pulsar Continue con todo vacío	Error: First Name is required	PASS	-
3	Completar First Name y pulsar Continue	Error: Last Name is required	PASS	-
4	Completar Last Name y pulsar Continue	Error: Postal Code is required	PASS	La URL siguió en checkout-step-one

Tipo	2 capturas de pantalla
Nombre o referencia	TC-04a_sin_first_name, TC-04b_sin_postal_code
Descripción	Validación del nombre obligatorio; Validación del código postal obligatorio
DESVIACIÓN ENCONTRADA	No aplica
REFERENCIA DEL DEFECTO	N/A
OBSERVACIONES	Ejecución automatizada con Playwright; resultado repetido en tres corridas.

Tabla 13. Registro de ejecución del caso TC-04
Fuente: Elaboración propia

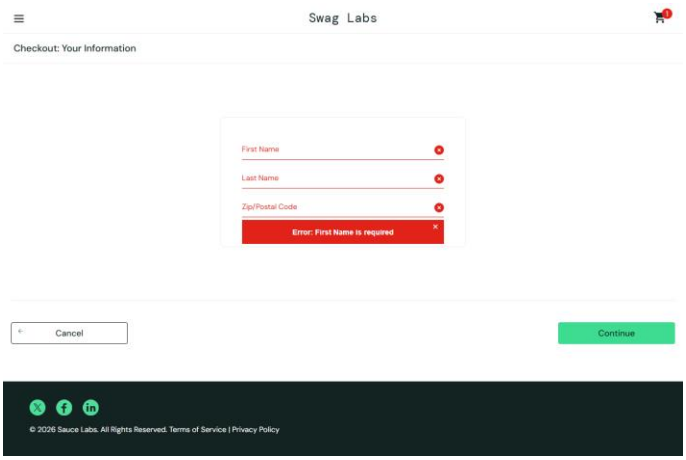


Figura 5. Validación del nombre obligatorio (TC-04)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

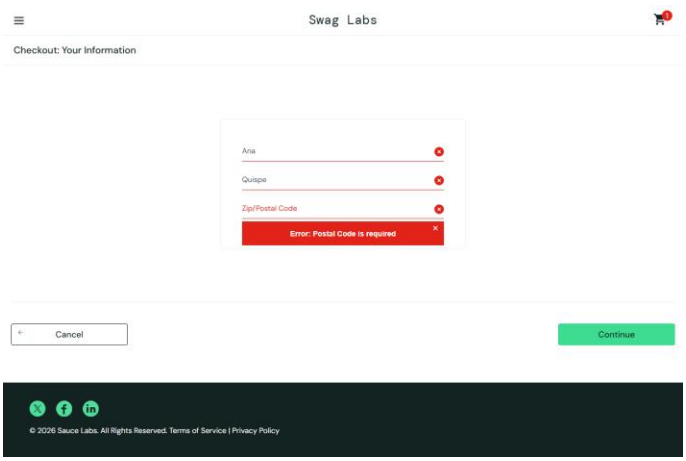


Figura 6. Validación del código postal obligatorio (TC-04)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

TC-05. Checkout con el carrito vacío

ID DEL TC	TC-05	**EJECUTADO POR**	Jhorddy Danner Condori Mamani
FECHA	20/10/2026	**HORA**	21:15
AMBIENTE	Producción pública (demo)	**VERSIÓN**	Publicada al 20/10/2026
☐ PASS	**☒ FAIL**	**☐ BLOCKED**	**☐ NOT RUN**
¿QUÉ OCURRIÓ REALMENTE?	El sistema permitió llegar al formulario, mostró un resumen con Total: $0.00 y confirmó la orden con «Thank you for your order!».		
COMPARACIÓN	No coincide. Se esperaba que la compra se bloqueara en el paso 2.		

#	**Paso ejecutado**	**Resultado observado**	**Estado**	**Observación**
1	Abrir el carrito sin agregar productos	El carrito apareció vacío	PASS	Checkout habilitado
2	Pulsar Checkout	Se abrió Checkout: Your Information	FAIL	No hubo aviso
3	Completar el formulario y pulsar Continue	Resumen sin productos, Total: $0.00	FAIL	-
4	Pulsar Finish	Thank you for your order!	FAIL	Orden vacía confirmada

Tipo	3 capturas de pantalla
Nombre o referencia	TC-05a_carrito_vacio, TC-05b_resumen_sin_productos, TC-05c_orden_completada_vacia
Descripción	Carrito sin productos con el botón Checkout habilitado; Resumen de compra sin productos con total $0.00; Confirmación de una orden vacía
DESVIACIÓN ENCONTRADA	La compra se completa sin productos en el carrito.
REFERENCIA DEL DEFECTO	BUG-01-F
OBSERVACIONES	Ejecución automatizada con Playwright; resultado repetido en tres corridas.

Tabla 14. Registro de ejecución del caso TC-05
Fuente: Elaboración propia

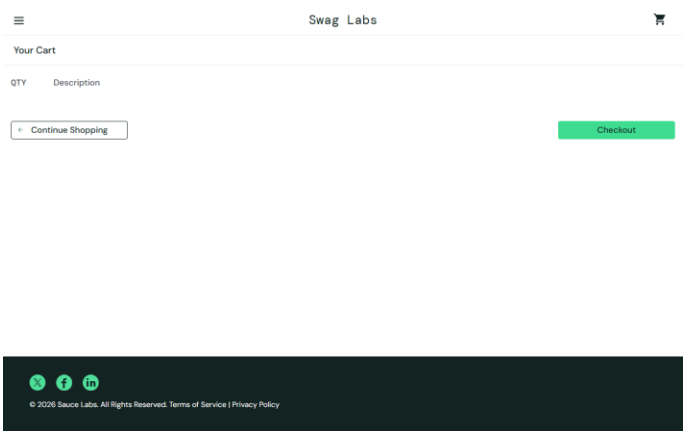


Figura 7. Carrito sin productos con el botón Checkout habilitado (TC-05)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

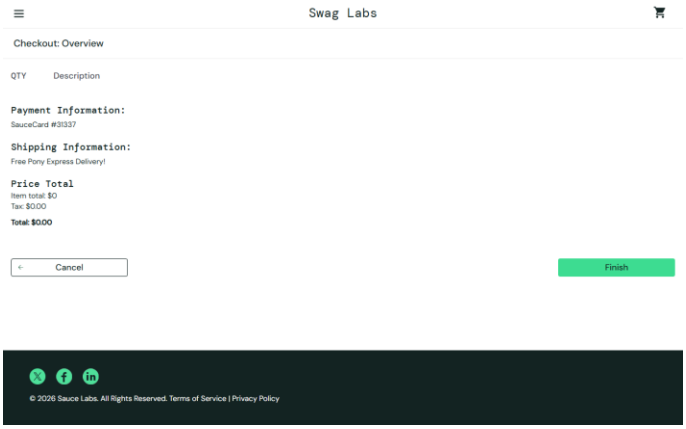


Figura 8. Resumen de compra sin productos con total $0.00 (TC-05)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

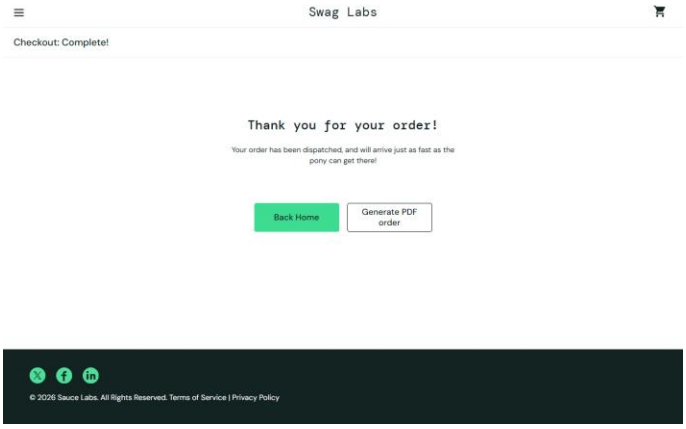


Figura 9. Confirmación de una orden vacía (TC-05)
Fuente: Captura de pantalla de SauceDemo durante la ejecución

4. Defectos y resultados

4.1. Resumen de ejecución

Indicador	Cantidad
Casos diseñados	5
Casos ejecutados	5
PASS	4
FAIL	1
BLOCKED	0
NOT RUN	0
Defectos encontrados	1

Tabla 15. Resumen de la ejecución de casos de prueba
Fuente: Elaboración propia

4.2. Defectos encontrados

ID	Título	Severidad	Prioridad	Estado	Caso
BUG-01-F	El sistema permite completar una compra con el carrito vacío	Alta	Alta	Abierto	TC-05

Tabla 16. Defectos detectados durante la ejecución
Fuente: Elaboración propia

Los casos que pasaron se registraron como PASS sin forzar hallazgos. El reporte completo del defecto, con pasos de reproducción y evidencia, se presenta en el apartado 4.6.

4.3. Métricas

Métrica	Fórmula	Valor	Interpretación
Tasa de ejecución	casos ejecutados / casos diseñados x 100	100 %	Todo el conjunto pudo ejecutarse; no hubo casos bloqueados
Tasa de éxito	casos PASS / casos ejecutados x 100	80 %	Cuatro de cinco comportamientos responden a lo esperado
Densidad de defectos	defectos / funcionalidades probadas	0,33	Un defecto en tres funcionalidades
Cobertura de criterios	criterios cubiertos / criterios definidos x 100	100 %	Los seis criterios de aceptación tienen al menos un caso

Tabla 17. Métricas obtenidas de la ejecución
Fuente: Elaboración propia

Las métricas se calculan sobre los cinco casos diseñados para las tres funcionalidades del alcance y no se extrapolan al resto de la tienda.

4.4. Matriz de trazabilidad

Funcionalidad	Criterio de aceptación	Caso	Resultado	Defecto
F-01	Usuario habilitado accede al catálogo	TC-01	PASS	-
F-01	Usuario bloqueado no accede y se informa	TC-02	PASS	-
F-02	Agregar y retirar actualiza lista y contador	TC-03	PASS	-
F-03	El formulario exige los datos obligatorios	TC-04	PASS	-
F-03	No se confirma una compra sin productos	TC-05	FAIL	BUG-01-F

Tabla 18. Matriz de trazabilidad entre criterios, casos, resultados y defectos
Fuente: Elaboración propia

4.5. Análisis de los resultados

El acceso respondió como se esperaba en sus dos variantes. El usuario habilitado llegó al catálogo y el usuario bloqueado quedó en el login con un mensaje que explica la causa, sin dejar dudas sobre por qué no pudo entrar.

El carrito mantuvo la coherencia entre la lista y el contador del encabezado. Al retirar un producto desde el carrito, el contador bajó de dos a uno sin recargar la página.

El checkout concentró el único resultado FAIL. La validación de los campos funciona: el formulario señaló, uno por uno, el nombre, el apellido y el código postal faltantes. El defecto no está en los campos sino en el flujo, porque el sistema nunca comprueba que exista algo que comprar. Una prueba del camino feliz, con productos en el carrito, lo habría pasado por alto; solo apareció al probar el límite inferior de la cantidad de productos.

4.6. Reporte del defecto BUG-01-F

#BUG-01-F	BUG REPORT
TÍTULO	El sistema permite completar una compra con el carrito vacío
DESCRIPCIÓN	En https://www.saucedemo.com/ el botón Checkout del carrito está habilitado aunque no haya productos. El flujo llega al resumen con total $0.00 y confirma la orden con «Thank you for your order!». La tienda acepta así órdenes sin contenido.
PASOS	1.- Ingresar a https://www.saucedemo.com/. 2.- Iniciar sesión con standard_user y secret_sauce. 3.- Sin agregar productos, abrir el carrito. 4.- Pulsar Checkout. 5.- Completar Ana, Quispe y 0000; pulsar Continue. 6.- Pulsar Finish. Referencia: TC-05
RESULTADO ACTUAL	El paso 4 abre Checkout: Your Information. El paso 5 muestra Checkout: Overview sin productos, con Item total: $0 y Total: $0.00. El paso 6 muestra «Thank you for your order!».
RESULTADO ESPERADO	El botón Checkout aparece deshabilitado con el carrito vacío, o el sistema avisa que falta agregar productos. Ninguna orden vacía se confirma.
PRIORIDAD	Alta 1-5 -> 2
SEVERIDAD	Alta 1-5 -> 2
MÓDULO	Carrito y checkout
FECHA DE DETECCIÓN	20 de octubre de 2026
AMBIENTE	Dispositivo = PC Sistema operativo = Windows 10 Pro Navegador = Chromium 156.0.8078.4 Versión del software = publicada al 20/10/2026
ADJUNTOS	Figura 7, Figura 8, Figura 9
COMENTARIOS	En una tienda real, el área de despacho procesaría órdenes vacías, se enviarían confirmaciones sin sentido y las métricas de ventas se distorsionarían. No impide la compra normal, por eso no se califica como crítica, pero afecta el proceso que genera ingresos. El defecto se reprodujo en tres de tres ejecuciones.
ESTADO	Abierto

Tabla 19. Reporte del defecto BUG-01-F
Fuente: Elaboración propia

5. Análisis de caja blanca

El fragmento entregado decide si una compra se aprueba según la edad del comprador, el estado de su cuenta, su saldo y el monto. Se numeraron las decisiones (D) y las sentencias de retorno (S) para referirlas en el análisis.

```
def validar_compra(edad, cuenta_activa, saldo, monto):
    if edad < 18:                          # D1
        return "Compra no permitida: edad"   # S1
    if not cuenta_activa:                  # D2
        return "Compra no permitida: cuenta" # S2
    if saldo >= monto and monto > 0:       # D3
        return "Compra aprobada"             # S3
    return "Compra rechazada"                # S4
```

5.1. Decisiones y condiciones simples

Decisión	Expresión	Condiciones simples
D1	edad < 18	C1: edad < 18
D2	not cuenta_activa	C2: cuenta_activa (D2 es verdadera cuando C2 es falsa)
D3	saldo >= monto and monto > 0	C3: saldo >= monto C4: monto > 0

Tabla 20. Decisiones y condiciones simples de validar_compra
Fuente: Elaboración propia

El código tiene tres decisiones, cuatro condiciones simples y ocho sentencias: los tres if y los cuatro return.

5.2. Criterio de cobertura

Se usa **cobertura de decisiones**: cada decisión toma al menos una vez su salida verdadera y su salida falsa. Este criterio incluye a la cobertura de sentencias, porque recorrer las dos salidas de cada if obliga a ejecutar los cuatro return. No debe confundirse con la cobertura de condiciones, que exige que cada condición simple, y no la decisión completa, tome ambos valores.

5.3. Mínimo de casos

Las tres decisiones van en secuencia y cada salida verdadera termina la función con un return. Por eso un mismo caso no puede cubrir dos salidas verdaderas:

- D1 verdadera termina en S1: necesita **1 caso propio**.
- D2 verdadera exige D1 falsa y termina en S2: necesita **1 caso propio**.

- D3 se alcanza solo con D1 y D2 falsas, y debe salir una vez verdadera (S3) y otra falsa (S4): necesita **2 casos**.

Mínimo para cobertura de decisiones = 1 + 1 + 2 = 4 casos. El resultado coincide con el número de caminos independientes (3 decisiones + 1 = 4) y con los cuatro return de la función.

5.4. Casos diseñados

Caso	edad	cuenta_activa	saldo	monto	Resultado esperado	Cubre
CB-01	16	True	500	100	Compra no permitida: edad	D1 = V (S1)
CB-02	25	False	500	100	Compra no permitida: cuenta	D1 = F, D2 = V (S2)
CB-03	25	True	500	100	Compra aprobada	D1 = F, D2 = F, D3 = V con C3 = V y C4 = V (S3)

Tabla 21. Tres casos de caja blanca diseñados
Fuente: Elaboración propia

Los tres casos se ejecutaron con pytest y pasaron. La cobertura que alcanzan se resume a continuación.

Criterio	Elementos cubiertos	Cobertura
Sentencias	7 de 8 (falta S4)	87,5 %
Decisiones	5 de 6 salidas (falta D3 = F)	83,3 %
Condiciones	6 de 8 valores (faltan C3 = F y C4 = F)	75 %

Tabla 22. Cobertura alcanzada con los tres casos
Fuente: Elaboración propia

La medición con coverage.py en modo de ramas lo confirma: de ocho sentencias queda una sin ejecutar (la línea 8, return "Compra rechazada") y de seis ramas queda una parcial. La salida completa está en evidencias/cobertura_caja_blanca.txt del repositorio.

5.5. Ajuste necesario

Los tres casos **no alcanzan** la cobertura de decisiones, porque D3 nunca sale falsa y S4 no se ejecuta. Se agrega el caso CB-04. Si se exigiera cobertura de condiciones, haría falta también CB-05. Python evalúa and con cortocircuito: cuando C3 es falsa, C4 no llega a evaluarse, y por eso CB-04 no basta para que C4 tome el valor falso.

Caso	edad	cuenta_activa	saldo	monto	Resultado esperado	Cubre
CB-04	25	True	50	100	Compra rechazada	D3 = F por C3 = F (S4). Completa decisiones
CB-05	25	True	500	0	Compra rechazada	C3 = V, C4 = F. Completa condiciones

Tabla 23. Casos que completan la cobertura
Fuente: Elaboración propia

Con CB-01 a CB-04 se llega a **100 % de sentencias y de decisiones**. Con los cinco casos, coverage.py reporta 100 % de sentencias y ramas, y cada condición simple toma ambos valores.

5.6. Valores límite

Una cobertura del 100 % no garantiza la ausencia de defectos. Si el código tuviera edad <= 18 en vez de edad < 18, los casos con edades 16 y 25 seguirían pasando con la misma cobertura. Ese tipo de error solo se detecta con valores límite:

Valor límite	Entrada	Resultado esperado
Edad justo por debajo	edad 17	Compra no permitida: edad
Edad en el límite	edad 18	Pasa a la validación de cuenta
Saldo igual al monto	saldo 100, monto 100	Compra aprobada
Monto mínimo positivo	saldo 100, monto 1	Compra aprobada
Monto negativo	saldo 100, monto -5	Compra rechazada

Tabla 24. Valores límite recomendados
Fuente: Elaboración propia

6. Escenarios Gherkin

Los escenarios describen, en lenguaje de negocio, el comportamiento esperado de dos funcionalidades probadas. Cada uno lleva la etiqueta del caso de prueba al que corresponde. Gherkin documenta el comportamiento; por sí solo no ejecuta las pruebas.

6.1. Inicio de sesión

```
Feature: Inicio de sesión en Swag Labs
  Como cliente de la tienda
  Quiero iniciar sesión con mi usuario
  Para acceder al catálogo de productos

  Background:
    Given que estoy en la página de login de "https://www.saucedemo.com/"

  @TC-01 @exitoso
  Scenario: Acceso exitoso con credenciales válidas
    When ingreso el usuario "standard_user"
    And ingreso la contraseña "secret_sauce"
    And presiono el botón "Login"
    Then debería ver la página "Products"
    And el ícono del carrito debería estar visible

  @TC-02 @error
  Scenario: Acceso rechazado para un usuario bloqueado
    When ingreso el usuario "locked_out_user"
    And ingreso la contraseña "secret_sauce"
    And presiono el botón "Login"
    Then debería ver el mensaje "Epic sadface: Sorry, this user has been locked out."
    But no debería acceder al catálogo
```

6.2. Checkout

```
Feature: Datos del comprador en el checkout
  Como cliente con productos en el carrito
  Quiero que el sistema valide mis datos de envío
  Para que la orden se registre correctamente

  Background:
    Given que inicié sesión con el usuario "standard_user"

  @TC-04 @error
  Scenario Outline: Checkout con un campo obligatorio vacío
    Given que agregué "Sauce Labs Backpack" al carrito
    And estoy en la página "Checkout: Your Information"
    When completo First Name "<nombre>", Last Name "<apellido>" y Postal Code "<código>"
    And presiono el botón "Continue"
    Then debería ver el mensaje "<mensaje>"
    And debería permanecer en la página "Checkout: Your Information"

    Examples:
      | nombre | apellido | código | mensaje                          |
      |        |          |        | Error: First Name is required    |
      | Ana    |          |        | Error: Last Name is required     |
      | Ana    | Quispe   |        | Error: Postal Code is required   |

  @exitoso
  Scenario: Compra completada con un producto
```

```
    Given que agregué "Sauce Labs Backpack" al carrito
    When completo el checkout con First Name "Ana", Last Name "Quispe" y Postal Code "0000"
    And presiono el botón "Finish"
    Then debería ver el mensaje "Thank you for your order!"
    And el carrito debería quedar vacío

  @TC-05 @BUG-01-F @error
  Scenario: No se permite comprar con el carrito vacío
    Given que mi carrito no tiene productos
    When abro el carrito
    Then el botón "Checkout" debería estar deshabilitado
    But no debería registrarse ninguna orden
```

6.3. Relación con los casos de prueba

Escenario	Caso	Resultado
Acceso exitoso con credenciales válidas	TC-01	PASS
Acceso rechazado para un usuario bloqueado	TC-02	PASS
Checkout con un campo obligatorio vacío (3 ejemplos)	TC-04	PASS
Compra completada con un producto	Flujo de referencia	Fuera de los 5 casos
No se permite comprar con el carrito vacío	TC-05	FAIL (BUG-01-F)

Tabla 25. Relación entre escenarios y casos de prueba
Fuente: Elaboración propia

7. Organización en Git y GitHub

El trabajo se versionó en un repositorio individual. Los archivos se crearon en local y se subieron con comandos de Git desde la terminal, con un commit por cada parte del examen.

7.1. Estructura del repositorio

Carpeta o archivo	Contenido
README.md	Propósito, URL probada, resumen de resultados e instrucciones para repetir la ejecución
docs/casos_caja_negra.md	Casos TC-01 a TC-05 con resultados
docs/analisis_caja_blanca.md	Decisiones, criterio, mínimo de casos y ajuste
docs/escenarios_gherkin.md	Escenarios BDD
bugs/bugs_encontrados.md	Reporte BUG-01-F
caja_blanca/	Código analizado y casos CB-01 a CB-05 en pytest
evidencias/	Capturas, script de ejecución y salida de cobertura
entregable/	Este informe

Tabla 26. Contenido del repositorio
Fuente: Elaboración propia

7.2. Comandos utilizados

```
git init -b main
git add README.md .gitignore
git commit -m "Estructura inicial del repositorio"
git add docs/casos_caja_negra.md evidencias/
git commit -m "Casos de caja negra ejecutados en SauceDemo con evidencias"
git add bugs/
git commit -m "Reporte de BUG-01: compra completada con el carrito vacío"
git remote add origin https://github.com/DannerCM/Examen-Hito-3-TQA.git
git push -u origin main
```

7.3. Issues y tablero

Las tareas y el defecto se registraron como ocho Issues en el repositorio y se organizaron en un tablero de GitHub Projects con las columnas To Do, In Progress, In Review y Done. El defecto lleva la etiqueta bug y enlaza su reporte y su evidencia.

Issue	Tipo
Diseñar casos de caja negra	Tarea
Ejecutar pruebas y registrar resultados	Tarea
Analizar código de caja blanca	Tarea
Redactar escenarios Gherkin	Tarea
Documentar bugs y revisar evidencias	Tarea
BUG-01-F: compra completada con el carrito vacío	Bug
Actualizar README	Tarea
Revisar y preparar informe PDF	Tarea

Tabla 27. Issues del tablero
Fuente: Elaboración propia

8. Conclusiones

El acceso y el carrito de SauceDemo responden como se espera. Un usuario habilitado entra al catálogo, un usuario bloqueado queda afuera con un mensaje claro, y el carrito mantiene la lista y el contador en sintonía al agregar y retirar productos. Los seis criterios de aceptación del alcance quedaron cubiertos por al menos un caso con evidencia.

Se encontró un defecto. El sistema permite completar una compra con el carrito vacío y confirma una orden de $0.00. La validación de los datos del comprador funciona; lo que falta es una regla que impida avanzar sin productos. El defecto apareció en un caso negativo diseñado sobre el límite inferior de la cantidad, lo que muestra el valor de probar más allá del camino feliz.

En el análisis de caja blanca, el mínimo para cobertura de decisiones de validar_compra es de cuatro casos, porque cada salida verdadera termina la función. Los tres casos pedidos dejan sin ejecutar el return "Compra rechazada". Con CB-04 se alcanza el 100 % de decisiones y con CB-05 el de condiciones, ya que el cortocircuito de and impide que un solo caso cubra ambas condiciones de la tercera decisión. La medición con coverage.py coincidió con el cálculo hecho antes de ejecutar.

Como limitación, las pruebas se ejecutaron en un solo navegador y sobre una aplicación de demostración cuya versión no se controla. Las métricas se refieren a las tres funcionalidades del alcance y no a la tienda completa.

9. Recomendaciones

- Deshabilitar el botón Checkout cuando el carrito está vacío y validar la misma regla en el servidor antes de registrar la orden.
- Repetir TC-05 como prueba de confirmación una vez corregido BUG-01-F, y ejecutar después la regresión de TC-03 y TC-04, que recorren el mismo flujo.
- Incorporar CB-04, CB-05 y los valores límite del apartado 5.6 a las pruebas de validar_compra.
- Ejecutar el mismo script en Firefox y WebKit para descartar diferencias entre navegadores.

10. Referencias

Cucumber. (2026). *Gherkin reference*. https://cucumber.io/docs/gherkin/reference/

International Software Testing Qualifications Board. (2023). *Certified Tester Foundation Level syllabus* (Version 4.0). https://www.istqb.org

Sauce Labs. (2026). *Swag Labs* [Aplicación web de demostración]. https://www.saucedemo.com/